

Processz ütemezés

Az előadás anyaga

Dr. Vadász Dénes: Operációs rendszerek
jegyzet/diasor és URL linkek alapján készült el

Témakörök

- Processz ütemezés fogalma
- Ütemezési szintek
- Ütemezési algoritmusok
- Mintapéldák

Processz ütemezés fogalma

„Ütemezés fogalma: processzek sorrendjének meghatározása.

Lényege: az erőforrás használatát egy OS komponens vezérli: az **ütemező** (scheduler).

Az ütemező felelős az *erőforrás kiosztásáért, elvételéért, nyilvántartásért* (döntés, végrehajtás).

Előnye: teljesen *kézben tartható* az erőforrás kezelése.

Hátránya: költségigényes megoldás (teljesítmény).

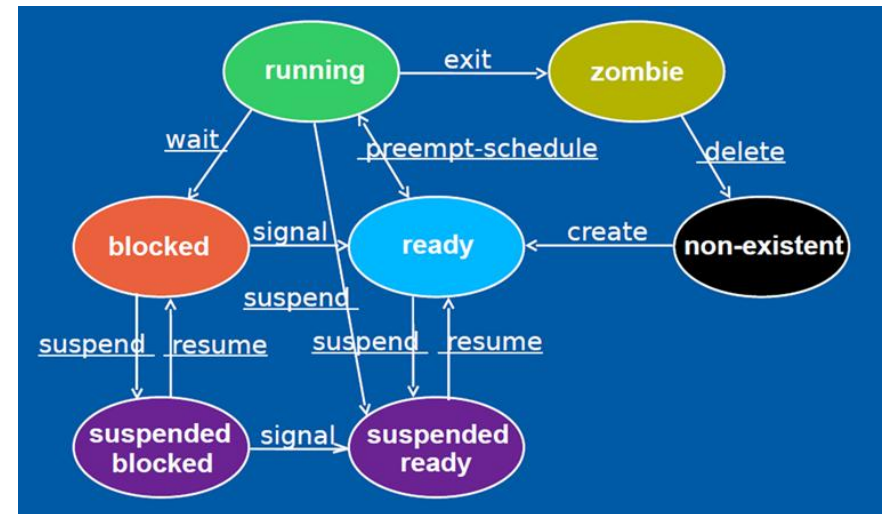
Processz ütemezés – ütemezési szintek

Három ütemezési szintet különböztetünk meg:

- Hosszútávú ütemezés.
- Középtávú ütemezés.
- Rövidtávú ütemezés.

Processz ütemezés – ütemezési szintek

- *Hosszútávú ütemezés: az elindítandó program kiválasztása.*
- *Középtávú ütemezés: felfüggesztendő és újra aktiválendő processzek kiválasztása.*
- *Rövidtávú ütemezés: a futásra kész processzek közül a következő futó kiválasztása.*



Processz ütemezés - long-term-scheduling

Ütemezési szintek

- *Long-term-scheduling* (hosszútávú)

*A **create** állapotátmenetért felelős.*

JOB ütemezés: processz pool-ból melyiket válasszuk ki futásra.

Processz ütemezés - medium-term-scheduling

Medium-term-scheduling (középtávú)

- A *suspended* (futásra nem kész) processzek közül melyeket helyezünk futásra készre (memória igény is érdekes).
- A ***suspend, resume*** állapotátmenetekért felelős.

Processz ütemezés - short-term scheduler

Short-term-scheduling (rövidtávú):

A ***preempt, schedule*** állapotátmenetekért felelős.

Kérdés az: hogy mikor *vegye el* a CPU-t a processztől, ill. a futásra kész processzek közül melyik kapja a CPU-t.

Az OS-k talán *legaktívabb komponense*, az **alacsony szintű ütemező** (rövid távú ütemező, low level scheduler, diszpécser).

Ütemezési algoritmusok alapjai

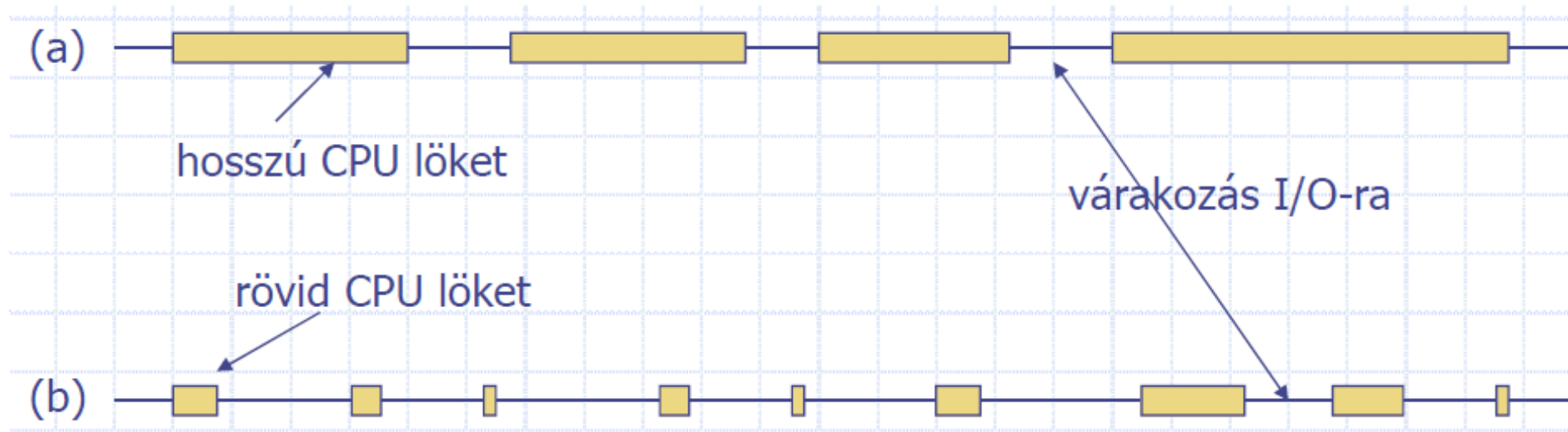
Egy processz futása során két különböző jellegű tevékenységet végez:

- *CPU löket (CPU burst) ideje alatt a **processz**-nek csak CPU-ra és az operatív tárra van szüksége.*
- *Periféria löket (I/O burst) alatt a **processz** perifériás átvitelt hajt végre, annak lezajlására várakozik.*

CPU löket és az IO löket

a.) CPU-korlátos processz

b.) I/O-korlátos processz



Processz ütemezés - elvárások

Elvárások a processz ütemezéssel szemben:

- *Pártatlanság:* minden processz (process, task, job) korrekt módon kapjon CPU-t.
- *Hatékonyság:* a CPU lehetőleg *legnagyobb százalékban* legyen kihasználva.
- *Válaszidő:* minimalizált válaszidő az *interaktív felhasználók számára* - elfogadható legyen.

Processz ütemezés - elvárások

- *Fordulási idő:* a processz *indulása és befejeződése között eltelt idő* – elfogadható legyen.
- *Teljesítmény:* időegységre eső processz feldolgozás jó legyen (más, mint a CPU kihasználás!)
- *Várakozási idő:* megadja, hogy a processz *mennyi időt töltött tétlen várakozással.*
- *Átfutási idő:* a processz *érkezésétől annak befejezéséig* eltelt idő.

Ütemezési algoritmusok teljesítményének mérése

1. CPU kihasználtság: $\Sigma \text{CPU-idő} - \Sigma \text{henyélés} / \Sigma \text{CPU-idő} * \%$

- *A hasznos munkával töltött idő aránya az összes időhöz képest.*

A CPU kihasználtságát a *kontextus váltáshoz*, ill. az *ütemezéshez szükséges idő* ronthatja le.

CPU kihasználtság (%): $t_{\text{CPU}} = t_{\text{CPU,munka}} + t_{\text{CPU,admin}} + t_{\text{CPU,idle}}$ (adminisztráció és a henyélés is bele tartozik), CPU kihasz. = $\text{Sum}(t_{\text{CPU,munka}}) / \text{Sum}(t_{\text{CPU}}) * 100 [\%]$

Ütemezési algoritmusok teljesítményének mérése

Kontextus váltás akkor történik, ha egy processzoron egy másik processz kezd el futni.

Ütemezés, amikor az ütemező algoritmusnak kell futnia.

Használatos képlet még:

$$\frac{\sum t_{CPU,hasznos\ munka}}{\sum t_{CPU}} \cdot 100 [\%]$$

Ütemezési algoritmusok teljesítményének mérése

2. Átbocsátó képesség : Adott időegység alatt elvégzett *feladatok* száma.

átbocsátó képesség: [munka/s] v. [1/s], adott időegység alatt elvégzett feladatok száma, rendszerfeladatokat nem számoljuk

$$\frac{\text{elvégzett munkák száma}}{\text{idő}} \left[\frac{1}{s} \right]$$

Ütemezési algoritmusok teljesítményének mérése

3. Várakozási idő : $\Sigma(\text{várakozó} + \text{futásra kész} + \text{felfüggesztett})/n$

- Egy *processz mennyi időt tölt várakozással* (futásra kész állapot, várakozó állapot, felfüggesztett állapotok).
- Az összes idő, amit a feladat várakozással töltött.
- Általában az *átlagos várakozási időt* számolunk.

$$t_{\text{várakozás}} = t_{\text{futásra kész}} + t_{\text{más, nem futó állapotok}} [s]$$

Ütemezési algoritmusok teljesítményének mérése

4. Körülfordulási idő: $\Sigma \text{CPU idő} + \Sigma \text{várakozás}/n$

- Egy processz a *rendszerbe helyezéstől a befejezésig eltelt idő*.
- Itt is az *átlagos körülfordulási időt* számolunk.

$$t_{\text{körülfordulás}} = t_{\text{CPU, végrehajtás}} + t_{\text{várakozás}} [s]$$

Ütemezési algoritmusok teljesítményének mérése

5. Válaszidő : Időosztásos (interaktív) rendszereknél fontos.

A feladat megkezdésétől az *első kimenetig* eltelt idő.

Itt is *átlagos válasz idővel* számolunk.

Processz ütemezés – stratégiák - ismételés

Ütemező döntési stratégiái (fejlődés)

1. *Nem beavatkozó* (non-preemptive)

a) *Run-to-completion* jellegű: egy processz futásának befejezéséig használhatja a CPU-t.

b) *Együttműködő* (cooperative): a processz saját belátása szerint dönt, hogy mikor mond le a CPU-ról.

Processz ütemezés - stratégiák - ismételés

2. Beavatkozó (*preemptive*): akkor is elvehetik a CPU-t egy processztől, ha az nem akar lemondani róla.

a.) Szelektív beavatkozó: bizonyos processzekbe nem avatkozik bele, másoktól meg elveszi a CPU-t.

b.) Beavatkozó: a CPU-t elveszi akkor is, ha nem mond le róla a processzek.

Ezek a *valódi időkiosztásos rendszerek*.

Processz ütemezés - stratégiák

3. *Nem preemptív*, egy processztől nem lehet elvenni a CPU-t.

- A processz csak az általa *kiadott utasítások hatására* vált állapotot.
 - erőforrásra, eseményre várakozás,
 - befejeződés vagy,
 - a CPU-ról önként lemondás.

Processz ütemezés - stratégiák

4. *Preemptív*: ha az OS elveheti a futás jogát egy processztől:

- *futásra kész állapotba* teszi a futó processzt és
- egy másik (futásra kész) processzt indít el.

Pl.: időosztásos, valós idejű OS-ek.

Mi prioritás és mi befolyásolhatja a kiválasztást?

A futásra kész processzek mindegyike kap egy számot, ami a futás sorrendjét határozza meg:

Prioritás = fontosság. Mi befolyásolhatja?

- A processzek „érkezési ideje” vagy „érkezési sorrendje”.
- A processzek memóriaigénye.
- Az eddigi CPU felhasználási idő.
- A várható CPU felhasználási idő.

Prioritás – típusai

A prioritás típusai:

- *belső:* a processz kéri, ill. az *operátor állítja be,*
- *külső prioritás:* az OS állítja be,

Ütemezéskor, *a legnagyobb prioritású kap futási engedélyt.*

Processz ütemezés – beavatkozás lehetőségei

Beavatkozás (preempt) lehetséges döntési algoritmusai:

1. *Időszeleten alapuló (időosztásos, time-sharing)*

A processz csak meghatározott *időszeletnyit* *futhat* egyhuzamban.

Az időszelet lejártával → *beavatkozás*.

Processz ütemezés - prioritáson alapuló

2. *Prioritáson alapuló*

A processzek rendelkeznek egy *fontossági értékkel* (prioritás).

Ha a „*ready*” állapotban levő processzek közül egynek „*magasabb*” lesz a prioritása, mint a „*running*” processz prioritása → akkor *beavatkozás*.

3. *Vegyes*

Processz ütemezés - prioritás

- *statikus prioritás*: a processz élete során időben állandó (csak speciális esetekben használatos)
- *dinamikus prioritás*: futási időben dől el melyik feladat és mikor fut - ezt használják.
- szükséges esetekben *újraszámolódik*, tehát az OS módosíthatja (időközönként, döntések előtt, sorokba bekerüléskor).

Prioritás *számító képlet* van.

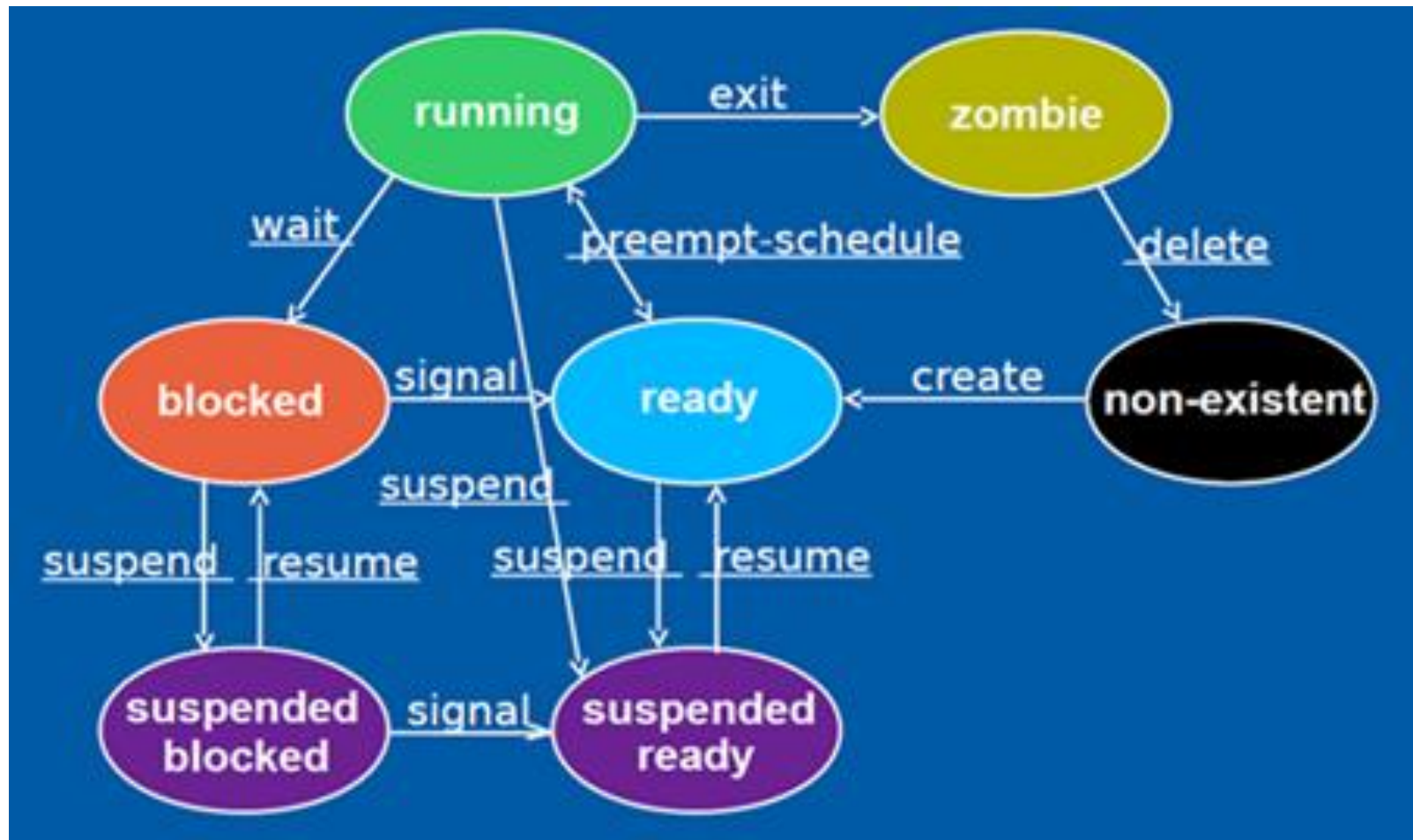
Processz ütemezés - Kiosztás

Kiosztás (schedule)

- Ha bármilyen okból felszabadult a CPU, akkor valamelyik *ready állapotúnak* meg kell kapnia.
- Mikor szabadul fel a CPU? : *wait, exit, preempt.*
- Az *algoritmus költsége* meghatározó a teljesítmény szempontjából, tehát ez is szempont.

Ütemezési algoritmusok

Ütemezési algoritmusok



Ütemezési algoritmusok

1. *Az igény-bejelentési sorrend* (First Come First Served, FCFS)

A processzek beérkezésük sorrendjében kapnak erőforrást, azaz ebben a sorrendben válnak futásra készké (ready)

2. *A legkisebb igényű először* (Shortest Job First)

A legrövidebb időigényű munkát szolgálják ki először.

3. *Körkörös sorrend* (Round-Robin Scheduling).

Ütemezési algoritmusok

4. *Igéret-vezérelt időkiosztás* (Policy Driven Scheduling)

A processzek rendszerben *eltöltött ideje* és az eddig *felhasznált CPU ideje* (cpu-idő) alapján.

5. *Többszintes prioritás-sorokon alapuló, kiosztási algoritmus* (Multilevel Queue Scheduling – MQS)

Ütemezési algoritmusok

6. *Többszintes prioritás-sorokon alapuló, visszacsatolásos kiosztási algoritmus* (Multilevel Feedback Queue Scheduling – MFQS)

7. *Többprocesszoros ütemezés.*

8. *Unix időkiosztása.*

Processz ütemezés algoritmusok

Klasszikus algoritmusok

1. **FCFS** (First Come, First Served)
2. **SJF** (Shortest Job First)
3. **RR** (Round-Robin)
4. **UNIX időkiosztás**

Processz ütemezés algoritmusok - FCFS

1. **FCFS** (First Come, First Served, Előbb jött, előbb fut)

- Nem preemptív algoritmus, I/O-ra várhat, a *nagyobb CPU löketű* processz *feltarthatja* a *kisebb löketű processzeket*.
- A *ready* állapotban lévő processzek a *beérkezési sorrendjük* szerint kapják meg a *CPU-t* és addig maguknál tarthatják, amíg *be nem fejeződnek*, vagy *egy periféria művelet miatt várakozni* nem kényszerülnek.
- Egyszerű a megvalósítás. (egy sor.)

Processz ütemezés algoritmusok - FCFS

Ezeket a processzeket *FIFO sorban (First Input First Output)* tárolják :

- `put ()` –tal bekerül a processz a várakozó sorba,
- `get ()` –tel levesszük a futásra kerülőt.
- Mivel *nincs beavatkozás*, kialakulhat a „*convoy effect*”.
- Nem garantált a megfelelő *fordulási idő, válaszidő*.

Processz ütemezés algoritmusok - FCFS

- Ha bekerül egy *nagy CPU löket idejű* processz a rendszerbe, akkor a processzek egész sorát tarthatja fel (*kamion hatás*).
- Az eljárás a *hosszú processzeknek* kedvez.
- A várakozási sor *legelső folyamatát futtatja*.
- **Nagy** az *átlagos várakozási idő* (konvoj hatás).

Processz ütemezés algoritmusok - SJF

2. **SJF** (Shortest Job First, Legrövidebb előnyben)

- Nem preemptív algoritmus.
- A processzek közül először a *legrövidebb löket idejű-t* választja futásra.

Előny: a legrövidebb *várakozási időt* adja.

Hátrány: egy *hosszú processz* sohasem kerül sorra, mindig lesz egy rövidebb, amelyik elé vág.

Processz ütemezés algoritmusok - SJF

Ez az ütemezési eljárás előnyben részesíti a *rövid processzoridő igényű* munkákat.

- *Ha több egyforma CPU idejű processz is lenne, közülük az futtat, amelyik előbb érkezett (FCFS).*

Processz ütemezés algoritmusok - SRTF

Az SJF algoritmusnak elképzelhető preemptív változata is, azaz ha egy processz futása közben érkezik egy nála rövidebb löketidő processz, azonnal megkapja a vezérlést.

- *SRTF (Shortest Remaining Time First) - ez az legrövidebb hátralévő idejű.*
- *Ilyenkor a környezetváltás is számolni kell!*

Processz ütemezés algoritmusok - RR

3. Round-Robin (RR, Körben járó, Körforgó)

- *Preemptív, időosztásos rendszerek alapja.*
- *Időszeleket* (quantum) rendelünk a processzekhez.
- *A ready állapotú processzek egy soron nyilvántartottak.*
- Minden processz, amikor futni kezd kap egy *időszeletet*.
- Egy processz futhat, amíg a időszelete tart (ha blokkolódik, akkor addig sem fut).

Processz ütemezés algoritmusok - RR

- Lejárt *szeletű* processz a **várakozási sor végére** kerül.
- A sor elején álló kapja a CPU-t. A listán a processzek “körben” járnak.

Előnye: egyszerű, kis költségű, garantáltan szóhoz jut minden processz.

Hátránya: nem vesz figyelembe prioritást(!).

Jelentős adminisztrációt igényel.

Processz ütemezés algoritmusok - RR

Érdekesség: a válaszidő csökkenthető, ha kisebb időszeletet használ az operációs rendszer, De!

- *Kisebb időszelet* → több CS → romló teljesítmény.
- *Nagy időszelet*: FCFS algoritmushoz hasonló lesz.

Ökölszabály: a CPU löketek kb. 80% -kal legyen rövidebb az időszeletnél. Ált.: 10-100 ms.

Mai napig is *gyakran használt algoritmus*.

Processz ütemezés – FCFS példa

Adott a következő FCFS algoritmusra.

Határozza meg:

- a.) Befejezési időt?
- b.) A P1...P5 processzek *várakozási/átlagos várakozási időt*?
- c.) Határozza meg a *processzek végrehajtási sorrendjét*?
- d.) Ábrázolja Gantt diagram segítségével az *aktív/várakozó* folyamatok futásának menetét?

Processz ütemezés – FCFS

FCFS megoldás

FCFS	P1	P2	P3	P4	P5
Érkezés	0	1	3	9	12
CPU idő	3	8	2	20	5
Indulás	0	3	11	13	33
Befejezés	3	11	13	33	38
Várakozás	0	2	8	4	21

Átlagos várakozás: 35/5

Végrehajtási sorrend: P1 - P2 - P3 - P4 - P5

Processz ütemezés – SJF példa

A processzek közül *először a legkevesebb* CPU időt igénylő fut. Ezért!

- Az SJF algoritmus garantálja a *legkisebb várakozási időt*, ill. optimális a *körülfordulási idő*.
- *Ha több egyforma CPU idejű processz is lenne, közülük az futhat, amelyik előbb érkezett (FCFS).*

Processz ütemezés – SJF példa

Adott a következő SJF algoritmusra.

Határozza meg:

- a.) A *várakozási/átlagos várakozási* időt?
- b.) A befejezési időt?
- c.) Ábrázolja Gantt diagram segítségével az *aktív/várakozó* folyamatok futásának menetét.

Megj.: a Gantt diagram ábrázolása szerkesztő program segítségével vagy Excel.

Processz ütemezés – SJF példa

SJF					
Processz	Érkezés	CPU idő	Indulás	Befejezés	Várakozás
F1	0	3	0		
F2	1	5	5		
F3	3	2	3		
F4	9	5	10		
F5	12	5	15		

Megj.: a Gantt diagram ábrázolása szerkesztő program vagy Excel segítségével.

Processz ütemezés – RR példa

Adott a rendszerben öt processz, *időszlet* 5 ms az alábbi paraméterekkel!

RR: 5 ms	Round Robin				
	P1	P2	P3	P4	P5
Érkezés	0	1	3	9	12
CPU idő	3	8	2	20	5
Indulás	0	3	8	13	18
Befejezés					
Várakozás					

Processz ütemezés – RR példa

RR: Időszelet - 5 ms

1. Határozza meg a

- a.) Ütemezze az adott időszelet alapján az egyes processzek paramétereit, határozza meg a befejezési és várakozási időt) (ms)!
- b.) A rendszerben lévő processzek végrehajtásának sorrendjét?
- c.) Ábrázolja Gantt diagram segítségével az *aktív/várakozó* processzek futásának menetét!

Megj.: a Gantt diagram ábrázolása szerkesztő program segítségével vagy Excel, Word programmal.

Processz ütemezés – RR példa

2. Határozza meg az *átlagos várakozási idő* és a *befejezési időt*?

RR: 5 ms	Round Robin				
	P1	P2	P3	P4	P5
Érkezés	0	1	3	9	12
CPU idő	3	8	2	20	5
Indulás	0	3	8	13	18
Befejezés					
Várakozás					

Processz ütemezés – RR példa

2a.) Határozza meg az átlagos *várakozási idő* és a *befejezési időt* és a *végrehajtási sorrendet*?

2b.) Ábrázolja Gantt diagrammal az *aktív/várakozó processzek* sorrendjét!

Teljesítménymérés - mintapélda

Feladat: Egy rendszerbe $P1....P4$ processzek érkeznek $0\ ms$ beérkezési idővel.

Határozza meg a *FCFS*, *SJF*, *RR: időszlet: 10 ms* algoritmusok esetén az alábbi teljesítmény jellemzőket:

	Processz	Érkezés(ms)	CPU idő (ms)
a.) CPU kihasználtság.	P1	0	9
b.) Körülfordulási idő átlaga.	P2	0	9
c.) Várakozási idő átlaga.	P3	0	9
d.) Válasz idő átlaga.	P4	0	9

Megj: az FCFS és SJF esetén a **CS** és az **ütemezési alg. idő:** 0.1 ms

Teljesítmény-mérés - mintapélda

FCFS algoritmus

Processz	9	18	27	36
P1				
P2				
P3				
P4				
Várk.:	P2, P3, P4	P3, P4	P4	-

Teljesítmény mérés	Értékek
CPU kihasználtság	4 cs, $(36,4 - 0,4) / 36,4 = 98,9 \%$
Körülfordulási idő átlaga	$(9+18+27+36) / 4 = 22,5$ ms
Várakozási idő átlaga	$(0+9+18+27) / 4 = 13,5$ ms
Válaszidő átlaga	$(0+9+18+27) / 4 = 13,5$ ms

Teljesítmény-mérés - mintapélda

SJF algoritmus

SJF algoritmus ütemezési *algoritmus lefutása ugyanaz*, mint az *FCFS esetén*, ebből adódik az is, hogy az algoritmust jellemző paraméterek is ugyan azok lesznek.

Processz ütemezés - Igéretvezérelt

4. Igéretvezérelt (Policy Driven)

- *Reális ígéret*: n számú processz esetén $1/n$ -ed részét kapja egy processz a CPU-nak.
- Mélni kell: az
 - *életidő*-t (a rendszerben eddig eltöltött idejét), és
 - *cpuidő*-t (eddig felhasználó CPU időt), számítandó a
 - *jogosultsági idő* = $\text{életidő} / n$;
 - *prioritás* = $\text{jogosultsági-idő} / \text{cpuidő}$.

Processz ütemezés - Igéretvezérelt

Más ígéretetek is lehetnek.

- Az *igéretvezérelt időkiosztás* speciális esete a *valós idejű* (Real Time) időkiosztás.

Hátránya: Nem vesz figyelembe fontosságot.

Processz ütemezés – Sorsjegyes

5. Sorsjegyes

A processzek sorsjegyeket kapnak, döntéskor sorshúzás, és annak nyertese kapja a CPU-t.

- Figyelembe vehető a *fontosság*: több sorsjegyet kap, nagyobb esélye van.
- Kisebb fontosságú processznek is van esélye.
- Csak „hosszabb” távon érvényesül a fontosság.
- Processz átadhat sorsjegyet, pl. gyerekeknek.

Processz ütemezés - Általános prioritásos

6. Általános prioritásos

- A *ready processzek* közül a legmagasabb prioritású kapja meg a CPU-t.
- Némelyik algoritmusnál az *alacsonyabb prioritás szám* jelent magasabb prioritást.
- A helyesen megválasztott prioritás számító képlettől függ, hogy teljesülnek-e az elvárások.

Processz ütemezés – prioritás függhet

A prioritás függhet:

- ***CPU használati idejétől*** (legfontosabb),
- a processz memóriaigényétől,
- erőforrás használati szokásaitól,
- a várható összes CPU használati idejétől,
- a rendszerben eddig eltöltött idejétől,
- külső prioritás értéktől,
- a rendszer terhelésétől.

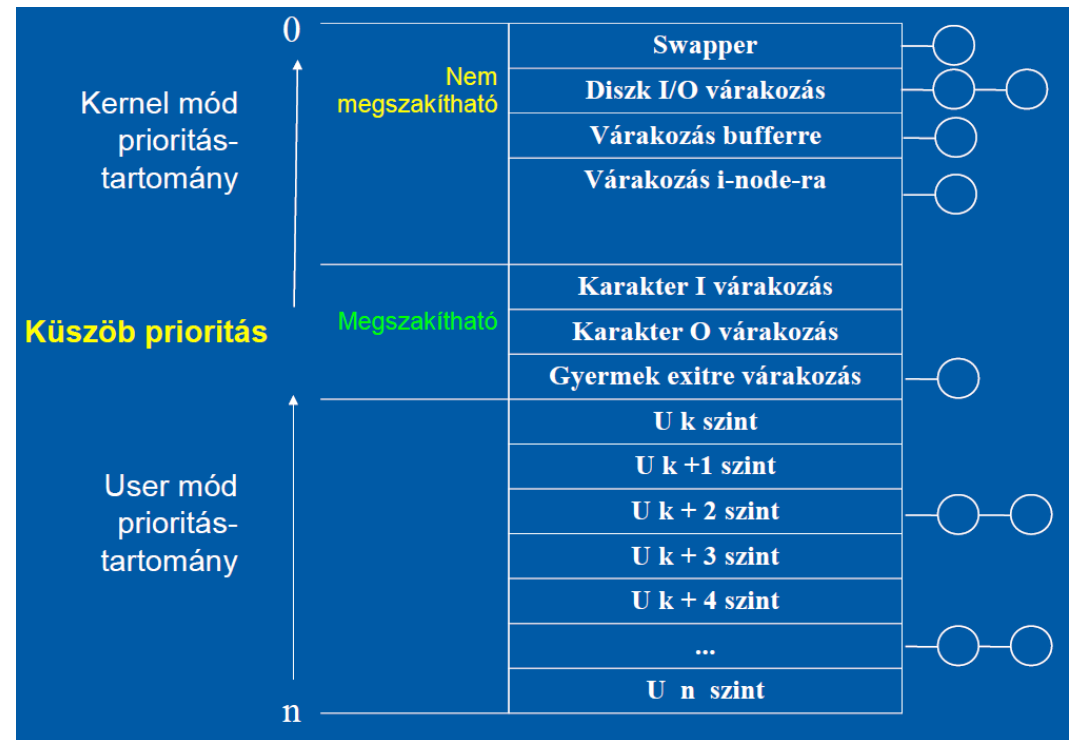
Processz ütemezés - UNIX időkiosztás

8. UNIX időkiosztás - *Multilevel Feedback Queue* jellegű.

- A processzeknek *user-mode* vagy *kernel-mode* *prioritásértéke* lehet.
- A *user-mode* a prioritás szintek *dinamikus a prioritás és beavatkozás*.
- A *kernel-mode* prioritás a *statikus*.

Processz ütemezés - UNIX időkiosztás

8. UNIX időkiosztás



Processz ütemezés - UNIX időkiosztás

8. UNIX időkiosztás

Prioritás számítás *(itt a **kisebb** érték jelenti a nagyobb prioritást!)*:

$$\mathbf{p\text{-}usrpri = PUSER + p\text{-}cpu / const2 + const3 * p\text{-}nice;}$$

(rendszerint $const2=const3=2$;

- A *p-usrpri* a küszöb érték alá nem mehet.
- A PUSER a processz *bázisprioritása*, ez *külső prioritás*.

Processz ütemezés - UNIX időkiosztás

Kernel a következő adatokkal írja le a folyamatokat:

- `p_pri` - a processzek aktuális prioritása.
- `p_usrpri` - a processz *felhasználói módban érvényes* prioritása.
- `p_cpu` - a korábbi CPU használat mértéke.
- `p_nice` - a felhasználó által *adott prioritás módosító* érték.

$$\text{p-}\underline{\text{usrpri}} = \text{PUSER} + \text{p-}\underline{\text{cpu}} / \text{const2} + \text{const3} * \text{p-}\underline{\text{nice}};$$

Processz ütemezés - UNIX időkiosztás

- A `p-nice` -- *a nice()* rendszerhívással (felhasználói kontrollt lehetővé tevő) beállított érték.
- A `p-cpu` -- a futó processznél a CPU használattal arányosan nő.
- A `p-cpu` -- *quantum-onként* öregbitik (aging):

$p\text{-cpu} = p\text{-cpu} / \text{const1};$ (rendszerint $\text{const1} = 2$)

$p\text{-usrpri} = \text{PUSER} + p\text{-cpu} / \text{const2} + \text{const3} * p\text{-nice};$

Processz ütemezés, Kiosztás

Egy példa

- Egy rendszerben a $PUSER=60$.
- Ha egy processz megkapja a CPU-t és quantum-ában végig használja, a p_cpu növekmény legyen 60.
- Legyen 3 processz (A, B, C), induláskor a C p_cpu -ja 7, a másik kettőé 0. Induláskor a p_uspri C-nél 63, a másik kettőnél 60.
- Mindhárom processznél a p_nice érték 0 legyen.
- Követhetjük, hogyan változnak a prioritások, melyik processz kap CPU-t.

Quantum	A processz		B processz		C processz	
	p_uspri	p_cpu	p_uspri	p_cpu	p_uspri	p_cpu
1	60	0	60	0	63	7
		1				
		2				
		...				
		60				7
2	75	30	60	0	61	3
				1		
				...		
		30		60		3
3	67	15	75	30	60	1
						2
						...
		15		30		61
4	63	7	67	15	75	30
		8				
		...				
		67		15		30
5	76	33	63	7	67	15
				8		
				...		

UNIX időkiosztás - felhasználói módban

- *Minden óraciklusban* a futó processzre növeli a `p_cpu` értékét

`p_pcu++`

- *Minden 100. óraciklusban* a kernel kiszámítja a prioritás értékét:

- minden folyamatnál „öregíti” a `p_cpu` értéket

`p_cpu = p_cpu * KF`

KF: korrekciós faktor „felejtés”

- kiszámolja a prioritást

`p_pri = P_USER + p_cpu / 4 + 2 * p_nice`

A `P_USER` konstans, a kernel és a felhasználói módot választja szét

`P_USER = 50`

Processz ütemezés - Linux scheduler

Az első változatok (v2 előtt) a *tradicionális ütemezőre* épültek.

a. *Linux $O(1)$ scheduler*

konstans idő alatt végzi el a döntést processzek számától függetlenül.

b. Linux CFS – Completely Fair Scheduler (teljesen korrekt ütemező)

$O(\log N)$ komplexitás "[1],[2],[3],[4],[5]"

Felhasznált irodalom

[1] Vadász Dénes: Operációs rendszerek, ME, 2007.

<http://www.iit.uni-miskolc.hu/~vadasz/GEIAL302B/2007-Ea>

[2] Kóczy, Kondorossi szerk: Operációs rendszerek mérnöki megközelítésben, Panem, 2000.

[3] Pannon Egyetem: Operációs rendszerek- ütemezés

https://dcs.uni-pannon.hu/files/docs/users/csertangyorgy/os/OS_05_Utemezes.pdf.

[4] Micskei Zoltán: Operációs rendszerek, BME.

[5] Mészáros Tamás: Unix folyamatok ütemezése, BME.

Taszk, szálak

Task, szál (fonál)

A modern OS-ek (és alkalmazásfejlesztők) támogatják a *konkurens programozást, a párhuzamosságot*.

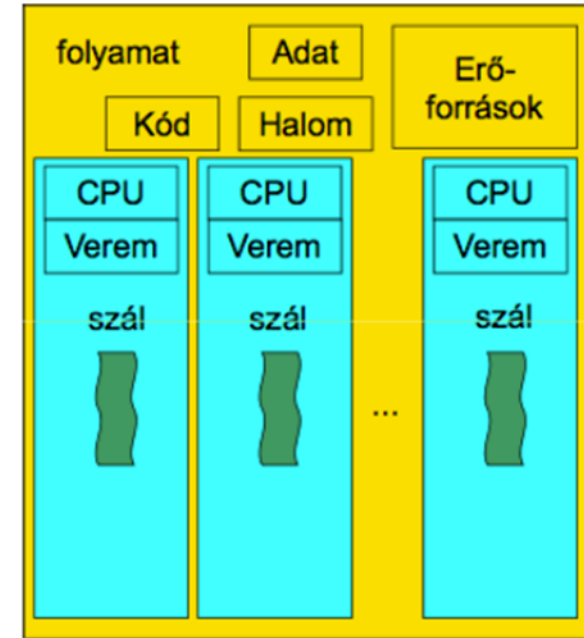
Ebben az esetben a *processz* fogalom helyett/mellett két másik fogalom: a *Fonál* (szál, thread) és a *Taszk* (Task).

A *futó program* ilyenkor a *taszk*, amely több szálat tartalmaz, amelyek a CPU-ért való vetélkedésben egymással is versenyeznek.

Szál jellemzők

Szál (fonál, thread, lightweight process: LWP):

- A CPU használat alapértelmezett egysége.
- Egy szekvenciálisan futó instrukciósorozat.
- Saját *virtuális CPU*-ja van, saját *veremtár*.
- *Kód, adat, halom, egyéb erőforrások, amelyek osztoznak a több szálakkal, melyekkel azonos processz kontextusában fut.*



Szál - jellemzők

- *Van dinamikus kontextusa* (regiszterkészlete, verme).
- Osztozik más szálakkal egy taszk statikus kontextusán (címkészlet: adat, kód stb.).
- A szálak valódi vagy pszeudó (ált. eljárások leírására használt mesterséges formális nyelv,) párhuzamosságban futhatnak.

Tehát,

- Folyamat = nehézsúlyú folyamat.
- Szál = pehelysúlyú folyamat.

Szálak - jellemzők

Feladat = Task = Folyamat = Process,

- Szál = Thread = Könnyűsúlyú folyamat
- Folyamat = Process = Nehézsúlyú folyamat
- Feladat = Task = Folyamat vagy Szál

Többszálas folyamatnál:

- szálakhoz van rendelve a CPU és minden szálnak saját stack-je van,
- a szálak osztoznak a processz közös adatán, kódján, heap-jén.

Szálak - jellemzők

Windows OS

- program vagy szolgáltatás = folyamat - ezen belül szálak,
- ütemező: szálakat ütemez.

Linux OS

- program vagy daemon = processz - ezen belül szálak,
- az ütemező taskokat ütemez, melyek lehetnek processzek vagy szálak.

Szálak létrehozása

Szálak létrehozása

- Win32: `CreateThread()` bonyolult paraméterezéssel.
- POSIX (Unix és Linux): Pthreads, kernel és user szinten is.
- Java nyelv:
 - VM a folyamat, VM-en belül szálak.
 - Thread osztályból származtatva, vagy Runnable megvalósítás.

Szálak - előnyei

Szálak előnyei

- Kis erőforrásigényű a létrehozás és megszüntetés.
- Applikáción belüli többszálúság - skálázhatóság (CPU magok kihasználása).
- Gyors kommunikáció közöttük (heap, adat, kód közös).

Task

Taszk

- *Van statikus kontextusa* (címkészlete, erőforrásai: nyitott I/O-k, adminisztrációs információk, szignálok stb.)
- *A végrehajtandó kódot* viszont a szálak tartalmazzák.
- Szál nélkül egy taszk passzív entitás.

Szálak kezelése – rendszerszintű

1. *Rendszerszintű (Kernel Level Threading)*

- Ezeket hívják (gyakran) *könnyűsúlyú processzeknek*.
- A kernel „*ismeri*” a szálakat, allokal nekik CPU-t, ütemezi őket, nyilvántartja állapotukat stb.
- A *szál* kontrollhoz (krealás, terminálás, stb.) *rendszerhívások biztosítottak*.
- Osztoznak a taszk *kontextusan, erőforrásaiban*.
- Viszonylag drága a menedzselésük (nyilvántartás, CS).

Szálak kezelése - felhasználói szintű

2. *Felhasználói szintű* (User Level Threading)

- A kernelnek nincs is tudomása a szálakról.
- A fejlesztő eszközhöz *adott RTL rutinokkal ütemezik*.
- Az RTL rutinok végzik a szál kontrollt (kreáció, megszüntetés stb.).
- Állapotukat is csak a *taszk tartja nyilván*.
- Osztoznak a taszk kontextusán, erőforrásain.

Szálak kezelése - felhasználói szintű

- Menedzselésük olcsó.
(nem kellnek rendszerhívások és CS)
- Természetesen felhasználói módban futnak.

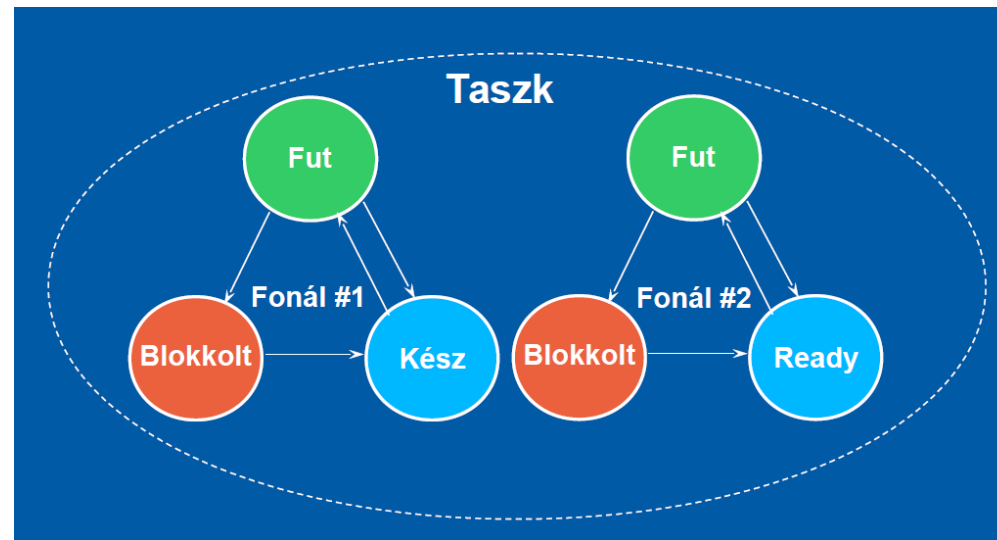
RTL: a programozónak futás idejű könyvtár rutin hívásnak tűnik (Run Time Library, RTL) - a futás idejű programkönyvtár.

- Információ átadás a paraméterekkel és a visszatérési értékkel (mint normál rutin hívás).

Task, Kernel szálak - állapotdiagram

A taszknak *nincsenek állapotai*, a szálaknak *viszont vannak*.

A szálak (LWP) ütemezését – akár több CPU-ra is – az OS scheduler végzi.



Task, szál

3. Vegyes:

a szál támogatás funkcionalitása megoszlik
(kernel/user)”[1],[2],[3],[4],[5]

Felhasznált irodalom

[1] Vadász Dénes: Operációs rendszerek, ME, 2007.

<http://www.iit.uni-miskolc.hu/~vadasz/GEIAL302B/2007-Ea>

[2] Kóczy, Kondorossi szerk: Operációs rendszerek mérnöki megközelítésben, Panem, 2000.

[3] Pannon Egyetem: Operációs rendszerek- ütemezés

https://dcs.uni-pannon.hu/files/docs/users/csertangyorgy/os/OS_05_Utemezes.pdf.

[4] Micskei Zoltán: Operációs rendszerek, BME.

[5] Mészáros Tamás: Unix folyamatok ütemezése, BME.